Vectorials Advisory Studio

Detailed Implementation Report

Submission Summary

This report documents the implementation of a multi-stage investment advisory application built with Next.js, TypeScript, Tailwind CSS, Zustand, Recharts, and Groq-based LLM tool calling. It covers the exact run instructions, the major architectural choices, key engineering challenges, and the most important improvements that would be implemented with additional time and resources.

Project Type:	Full-stack advisory workflow application
Frontend:	Next.js, React, TypeScript, Tailwind CSS
Backend:	Next.js API routes, TypeScript
Inference Provider:	Groq
Model:	`llama-3.1-8b-instant`
Date:	March 26, 2026

Contents

1. Executive Summary

Vectorials Advisory Studio is a five-stage advisory workflow application designed to simulate how a modern wealth advisory product could collect client information, parse investment documents, assess risk, generate recommendations, score opportunities, and present them in a dashboard.

The implementation emphasizes four goals:

- a polished end-to-end user experience,
- real multi-agent LLM orchestration,
- proper server-side tool calling instead of mocked AI behavior,
- privacy-aware preprocessing before model analysis.

The final design combines deterministic logic and LLM reasoning. Deterministic code handles parsing, redaction, chunking, scoring, and portfolio math. The LLM layer focuses on interpretation, synthesis, and recommendation generation. This split reduces hallucination risk, lowers token usage, and produces a more reliable demo.

Key Outcome

The final system uses Groq with real tool calling for all three agents, supports local file upload and sanitization, runs in a single Next.js codebase, and can be started locally with a small environment setup.

2. How To Run The Application

2.1. Prerequisites

Before running the application, the following are required:

- Node.js 20 or newer recommended
- npm
- a Groq account
- a Groq API key

2.2. How To Generate A Groq API Key

The project uses Groq as the model provider. To create a key:

1. Open the Groq console at https://console.groq.com.
2. Sign in or create an account.
3. Navigate to the API key page at https://console.groq.com/keys.
4. Click `Create API Key`.
5. Copy the generated key.
6. Keep the key private and do not commit it to version control.

2.3. Environment Setup

Create a file named `.env.local` in the project root.

The contents should be:

```
GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL=llama-3.1-8b-instant
```

2.4. Development Run

From the project root, install dependencies and start the development server:

```
npm install
npm run dev
```

Then open the application in a browser:

```
http://localhost:3000
```

2.5. Production Build Verification

To verify the production build locally:

```
npm run build
npm start
```

2.6. Recommended Demo Flow

The repository contains example data for a clean demonstration:

- `example_portfolio.csv`
- `example_client_brief.txt`

Recommended demo steps:

1. Upload `example_portfolio.csv` in Stage 1.
2. Use the client brief as the narrative context for profile and assessment answers.
3. Complete the hybrid questionnaire in Stage 2.
4. Run the analysis pipeline in Stage 3.
5. Review scores in Stage 4.
6. Inspect the dashboard and export in Stage 5.

2.7. Useful Notes During Local Development

- The Stage 3 pipeline depends on a valid Groq API key.
- The application keeps sessions in memory for the active runtime.
- On Windows, a `.next\trace` file lock can occasionally block rebuilds. If that happens, stop any running Next.js process, delete the `.next` folder, and rebuild.

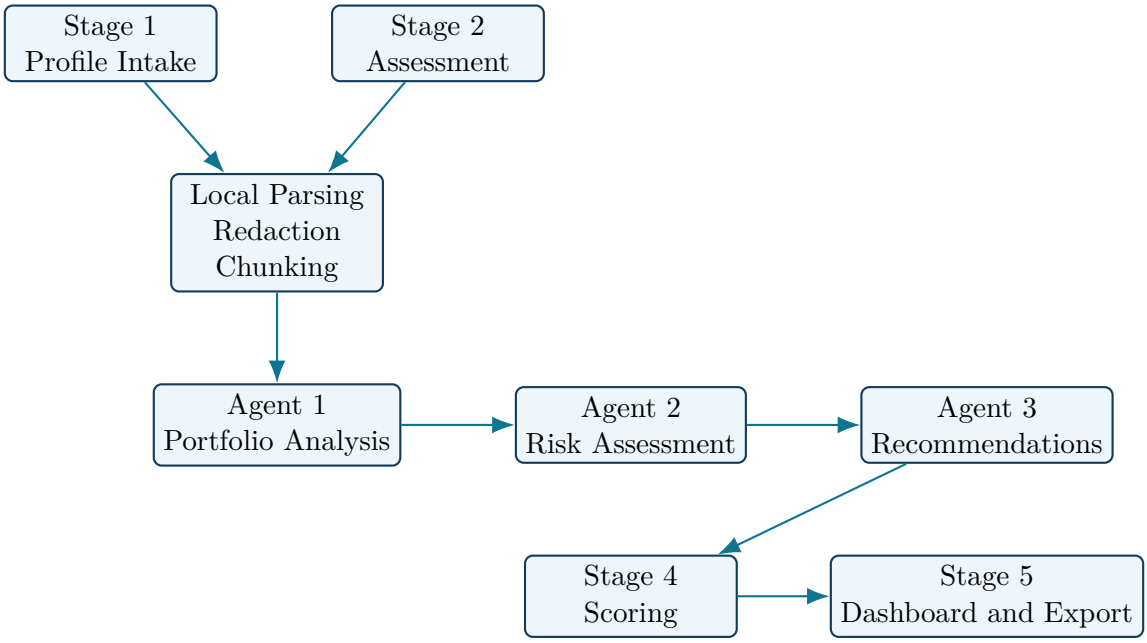
3. System Overview

3.1. Functional Workflow

The application is divided into five stages:

1. **Profile Intake** Capture client information and upload portfolio files.
2. **Assessment** Ask baseline questions and dynamic follow-up questions.
3. **Analysis** Run three sequential agents with scoped tool access.
4. **Scoring** Compute feasibility and impact scores and summarize the result.
5. **Dashboard** Present recommendations, compare them visually, and export results.

3.2. Architecture Diagram



4. Key Architecture Decisions

4.1. Single Full-Stack Next.js Codebase

A major decision was to use Next.js for both the interface and the server-side orchestration layer. This provided several benefits:

- UI and API logic live in one repository.
- Sensitive LLM calls stay server-side.
- File parsing and redaction can be executed close to the analysis pipeline.
- The project is easier to run locally with a simple `npm install` and `npm run dev`.

This choice reduced coordination overhead compared with splitting the app into a React frontend and a separate Python or Node backend.

4.2. TypeScript End To End

TypeScript was used as the primary language across frontend, backend, agent contracts, store state, and utility functions.

This was important because the project includes:

- multi-stage workflow state,
- structured agent outputs,
- tool schemas,
- portfolio and recommendation domain models,
- export and scoring logic.

Using TypeScript helped keep the contracts between UI, APIs, and agent results explicit and easier to validate.

4.3. Hybrid Deterministic Plus LLM Design

A critical architecture decision was to avoid giving the LLM responsibility for every task.

Instead:

- deterministic code handles parsing, sanitization, chunking, risk indicators, scoring, and portfolio math,
- the LLM handles interpretation, summary generation, and advisor-style recommendation synthesis.

This split improves reliability because:

- sensitive calculations do not depend on probabilistic output,
- prompt size stays smaller,
- token consumption is reduced,
- outputs remain more consistent across runs.

4.4. Sequential Three-Agent Design

The system uses three agents in sequence rather than a single all-purpose model invocation.

4.4.1. Agent 1: Portfolio Analysis

This agent focuses on:

- reading sanitized portfolio context,
- inspecting document chunks when needed,
- identifying diversification and concentration patterns.

4.4.2. Agent 2: Risk Assessment

This agent focuses on:

- concentration risk,
- liquidity and compliance-style checks,
- risk interpretation relative to client goals.

4.4.3. Agent 3: Recommendation Generation

This agent focuses on:

- generating recommendations,
- rebalancing logic,
- implementation cost and tax-aware transition framing.

The sequential design improves traceability. It also maps well to the UI, where users can understand progress and results by stage.

4.5. Scoped Tool Access Per Agent

Each agent is given only the tools it should use. This is important for both control and explainability.

Examples:

- Agent 1 can inspect sanitized document chunks.
- Agent 2 can read portfolio and Agent 1 outputs, but not arbitrary raw data.
- Agent 3 uses recommendation and transition-impact tools instead of raw document access.

This design reduces unnecessary context exposure and aligns well with a least-privilege model.

4.6. Local Parsing, Redaction, And Chunking

Another important decision was to preprocess uploaded files before any model call.

This preprocessing pipeline:

- parses the uploaded file locally,
- extracts text,
- removes or masks likely sensitive identifiers,
- chunks long text into manageable units,
- prepares a structured portfolio snapshot.

This helps solve both privacy and context-window problems.

4.7. Groq As The Final Inference Provider

Several provider options were considered and tested conceptually during the project. Groq was selected as the final provider because it balanced:

- cost,
- latency,
- tool-calling compatibility,
- ease of integration,
- free-tier accessibility for development.

The project uses the OpenAI-compatible SDK pattern pointed at Groq's API base URL, which kept the implementation straightforward while still enabling real tool calling.

4.8. Frontend State With Zustand

The application spans multiple stages, multiple forms of input, asynchronous analysis operations, and dashboard output. Zustand was chosen because it provides:

- a simple mental model,
- minimal boilerplate,
- good fit for wizard-style workflows,
- easy persistence of session-level UI state during navigation.

4.9. CSV Export As A Reliability-Focused Deliverable

CSV export was chosen instead of custom PDF report generation in the initial version. This was a deliberate scope decision.

Reasons:

- CSV is simple and reliable,
- it satisfies the output requirement,
- it keeps engineering effort focused on the advisory workflow and analysis pipeline,
- it avoids spending too much time on printable layout logic.

5. Challenges Faced And Solutions

5.1. Challenge 1: Free Tier API Constraints And LLM Provider Experiments

One of the most important challenges was finding a provider that supported real tool calling while remaining practical for a student or candidate demo environment.

A multi-agent application is expensive by nature because it tends to involve:

- multiple sequential model calls,
- tool-calling loops,

- structured output generation,
- repeated iteration during development and debugging.

5.1.1. Experiments And Reasoning Across Providers

Provider	Observed Strength	Main Issue	Outcome
OpenAI	Strong tool-calling ecosystem and mature API design	No dependable free tier for repeated development; billing is separate from ChatGPT; cost concern for iterative testing	Not selected as the final provider
Gemini	Available free-tier access and generally capable models	Free quota was exhausted quickly in a three-agent flow; provider-specific function schema differences also created additional integration friction	Used as an experiment, then replaced
Groq	Fast inference, free plan, OpenAI-compatible integration style, good fit for compact tool-calling workflows	Tight token and rate budgets still require aggressive prompt and payload discipline	Selected as the final provider
Local LLM via Ollama	No per-request API billing and appealing privacy story	Setup burden, hardware dependence, weaker tool-calling consistency, and less reliable demo experience	Deferred for future work

5.1.2. Solution

The solution was not merely to switch providers. The final solution combined provider choice and architectural optimization:

- Groq was selected as the final provider.
- The default model was set to `llama-3.1-8b-instant`.
- Prompts were kept concise.
- Agent outputs were constrained to compact structured JSON.
- Long raw document content was not repeatedly sent to later agents.
- Deterministic preprocessing and scoring logic were moved out of the LLM path.

This significantly improved free-tier viability while preserving real model behavior.

Challenge Highlight: Free Tier API Pressure

The most visible challenge in the project was not building the UI, but making a real multi-agent, tool-calling pipeline work under free-tier constraints. The solution required both technical changes and provider changes. Reducing tokens, limiting tool loops, and choosing Groq over Gemini for the final system were all part of the same engineering decision.

5.2. Challenge 2: Provider-Specific Tool Calling Differences

Tool calling is not standardized across providers. During experimentation, the same conceptual tool definitions behaved differently depending on the API.

Examples of friction included:

- Gemini expecting different function declaration shapes than OpenAI-style tools,
- validation failures when no-argument tools were defined too strictly,
- model-specific behavior around extra fields being included in tool-call arguments.

5.2.1. Solution

The solution was to tighten the implementation around a single final provider and explicitly shape the tools for that provider's expectations.

The final Groq implementation:

- uses OpenAI-compatible tool definitions,
- allows permissive no-argument parameter objects where appropriate,
- keeps required-argument tools strict,
- validates structured outputs after model generation.

This reduced runtime surprises and improved the reliability of Agent 3 in particular.

5.3. Challenge 3: Long Financial Statements And The "Lost In The Middle" Problem

Financial statements can be long and noisy. Sending an entire document into a single prompt risks:

- high cost,
- lower retrieval quality,
- missing important middle sections,
- weaker recommendation quality.

5.3.1. Solution

The project introduced a preprocessing pipeline:

- parse documents locally,

- sanitize them,
- split them into chunks,
- store chunk metadata,
- let Agent 1 inspect chunks through a tool when needed.

This means the agent does not read the entire raw statement blindly. Instead, it accesses relevant summarized chunks through controlled tool calls.

5.4. Challenge 4: Protecting Confidential Data During Analysis

A financial advisory workflow must avoid sending confidential personal details directly to the LLM whenever possible.

The risk areas included:

- names,
- contact details,
- account numbers,
- tax identifiers,
- long raw statement text.

5.4.1. Solution

The implemented solution was a least-privilege data strategy:

- parse locally,
- redact likely identifiers,
- expose structured facts through tools,
- avoid sending raw documents into downstream agent stages,
- keep calculations server-side.

This approach is not full enterprise compliance, but it is a strong architectural step toward safer model integration.

5.5. Challenge 5: Multi-Stage Workflow State Management

The application is not a single form. It is a staged workflow with:

- uploads,
- structured answers,
- dynamic question routing,
- asynchronous agent progress,
- recommendation scoring,

- dashboard interactions.

Without careful state design, the UI would become fragile.

5.5.1. Solution

A centralized Zustand store was used to:

- preserve workflow state across stages,
- isolate step transitions,
- manage prepared analysis data and agent outputs,
- support a coherent dashboard experience.

This kept the user journey consistent while avoiding excessive state boilerplate.

5.6. Challenge 6: Balancing Quality Against Delivery Risk

There were several tempting features that could have consumed too much time:

- fully custom PDF reporting,
- advanced OCR or complex PDF table extraction,
- deep personalization using larger prompt contexts,
- building for every provider at once.

5.6.1. Solution

The project stayed disciplined:

- one full end-to-end working product was prioritized,
- CSV export was chosen for reliability,
- the AI layer was kept real but bounded,
- the UI was polished without depending on hidden backend assumptions.

6. What I Would Improve With More Time

6.1. Move To A Paid Tier For Greater Demo And Production Reliability

The first improvement would be to move the project from a free-tier development assumption to a paid provider tier.

Why this matters:

- free tiers are useful for prototyping but unpredictable for repeated demos,
- paid usage allows higher rate limits and better operational confidence,
- it reduces the need to over-optimize every prompt and payload for quota survival.

Practical upgrades under paid tier:

- allow richer document analysis,
- support larger recommendation narratives,
- add retry logic with fewer concerns about quota exhaustion,
- run deeper evaluation and regression testing across more sample portfolios.

6.2. Run Open-Source LLMs On Cloud Infrastructure Such As AWS

Another major improvement would be to support a self-hosted or semi-managed open-source model deployment on cloud infrastructure.

An AWS-based direction could include:

- EC2 GPU instances with an inference server such as vLLM,
- ECS or EKS for containerized model hosting,
- S3 for encrypted document storage,
- CloudWatch for observability,
- a gateway layer for model routing and fallback.

Why this would help:

- lower long-term dependency on a single third-party inference API,
- more control over data residency,
- easier experimentation with open-source model families,
- potential cost advantages at scale,
- better support for provider abstraction in the long term.

This would be especially useful if the project evolved into a privacy-sensitive enterprise product.

6.3. Introduce A True Provider Abstraction Layer

The current implementation is clean, but it is still optimized around the final Groq decision. With more time, I would formalize a provider abstraction with:

- a shared `LLMProvider` interface,
- separate adapters for Groq, OpenAI, Gemini, and local inference,
- a unified tool schema conversion layer,
- environment-driven provider selection.

This would make provider experiments safer and reduce migration effort in the future.

6.4. Improve Document Parsing And Portfolio Extraction

Current PDF support is text-extraction based. That is sufficient for a candidate assignment, but not ideal for production.

I would improve this by:

- adding stronger table extraction,
- handling broker statement layouts more explicitly,
- supporting OCR for scanned PDFs,
- building a normalization layer for holdings and transaction formats.

This would improve both agent quality and dashboard trustworthiness.

6.5. Add Persistent Storage And Auditability

At the moment, analysis sessions are stored in memory only.

With more time, I would add:

- a database for client sessions and recommendations,
- durable storage for uploaded files,
- versioned recommendation records,
- tool-call audit trails,
- role-based access to recommendation history.

This would move the project from demo-grade workflow to operational product foundation.

6.6. Strengthen Security And Compliance Readiness

A more mature version of the project should include:

- encrypted storage at rest,
- authenticated access,
- secrets management,
- structured redaction policies,
- clearer retention and deletion rules,
- better logging of what is and is not sent to the model.

This would be essential if the application were to handle real client data.

6.7. Improve The User Experience With Streaming And Better Explainability

The current UI already presents staged progress, but there is room to make the experience more transparent.

Potential improvements:

- live streaming updates while tools are being called,
- richer intermediate summaries,
- deeper rationale panels for each recommendation,
- visual traceability from finding to recommendation to score,
- editable advisor notes before export.

This would make the product more useful for both demo scenarios and real advisor workflows.

6.8. Build An Evaluation Harness

A serious next step would be to create a repeatable evaluation suite with:

- known input portfolios,
- expected concentration findings,
- expected recommendation patterns,
- JSON schema validation tests,
- prompt and provider regression tests.

This would improve confidence when changing prompts, tools, or providers.

6.9. Generate Polished PDF Reports In Addition To CSV

CSV export is reliable, but advisors often want a printable deliverable. A future version should add:

- client-ready PDF summaries,
- branded report layouts,
- charts embedded into export output,
- recommendation rationales in narrative form,
- appendices for risk and implementation notes.

7. Conclusion

This implementation focused on building a credible advisory workflow end to end rather than over-investing in isolated features.

The most important achievements were:

- a complete five-stage product flow,
- real three-agent analysis with tool calling,
- privacy-aware preprocessing,

- a clean TypeScript full-stack implementation,
- a provider decision that made development practical under real-world constraints.

The main engineering story of the project is that success did not come from simply connecting an LLM API. It came from structuring the system carefully:

- deterministic code where precision matters,
- LLM reasoning where synthesis matters,
- scoped tools for control,
- budget-aware design for free-tier viability,
- UI choices that make the workflow understandable and trustworthy.

If extended further with paid-tier infrastructure, stronger parsing, persistent storage, and a provider abstraction layer, this project could evolve from a candidate assignment submission into the foundation for a more production-ready advisory platform.